vesuvius-challenge-surface-detection

# 2nd Place Solution Vesuvius Challenge – A postprocessing win

| | |
|---|---|
| **Rank** | 2 |
| **Team** | risk of overfitting |
| **Author** | Marius Heuser |
| **Topic ID** | 679278 |
| **Version** | Original |

First of all, we want to thank kaggle and the organizers for this interesting and fun competition. A special thank you to @giorgioangelotti and @seanjohnsonsp for being great hosts. Eventhough there has been negative feedback in some regard, I was impressed and very happy with the constant communication, even if difficult decission had to be made. That is not a given, so thank you alot!

# Overview:

We used nnUNet to train $128^3$ and $160^3$ patchsize models and ensembled them with a 40%/60% weighting in favor of the $160^3$ model. The postprocessing consists of local componentwise interpolation with multiple segmentation masks (different thresholds).

## Detailed model description:

#### Single Model Strategy

We trained two models on vanilla nnUNet ResEncTrainer with the following settings:

patch size: 128 batch size: 2 epochs: 1000 fold: all

patch size: 160 batch size: 2 epochs: 1000 fold: all

#### Ensemble Strategy

We summed the logits with a weighting of 60% of $160^3$ model and 40% of $128^3$ model. This was just visually tuned, since the $160^3$ model scored higher than the other one.

#### Model history:

We started with a $128^3$ model trained on 80% of the training data. We used this model for tuning the thresholds for the final $128^3$ model, which seemed to translate well. We finished training the full $160^3$ model in the last days of the competition and the thresholds from the $128^3$ model didn't apply to it, so we didnt have time to tune the thresholds for the ensemble. It was mostly done on a visual basis and a few LB submissions.

For a really long time we tried to come up with a better model by changing the loss function. I invested alot of time to implement the „Skeaw-BorT“-Loss from a paper I found, just to realize that it doesn't work here, since the background can't be split into components. The medialsurface loss also didn't help for the following reason: The resulting model would perform better on topo score but its performance on surface dice would decrease. We had plenty ideas to increase the topo score by postprocessing, so it was way more important to have models, which score as high as possible on surface dice, than having models which were decent at everything.

## Post-processing:

After ensembling the model logits (mirroring+rotation TTA), we created multiple segmentation masks with the following postprocessing structure:

1. topo_postprocess: Consisting of 3d Hysteresis, 3d Anisotropic Closing, Dust Removel This is the same postprocessing, which can be found in the public workbooks.

2. Set the 3 voxels of each volume face to 0 (these are label 2 voxels)
3. Dust removel again (size 1000)

The reason for step 2 and 3 are for special cases, where after removing label 2 it could create artifacts. This was a known issue in this competition, but we didnt have any information on label 2 positions other than the 3 voxel at each volume face.

In the highest final submission score, we used 4 thesholds for topo_postprocess:

The base segmentation: T_low: 0.2

The fine tuning segmentations: T_low: 0.5 T_low: 0.6 T_low: 0.7

After that we performed binary_fill_holes on the base segmentation and cropped the volumes by removing the 3 voxels of each volume face for every segmentation. Why didnt we do this right away? This step was implemented way later and wasnt important for the submitted solution, I will explain it in the history section or a comment later. I had a eurika moment, when i realized the tunnel/holes can be quickly located (the existence) by calculating the euler number:

euler

(C=1 (components), H=0 (cavities), T=#Holes/Tunnels)

Now we performed the following steps:

1. Take the base segmentation and split it into volumes with just one component
2. Window-check small patches of each volume for tunnel/holes by calculating the euler number
3. a) Euler number >= 1 → save component segmentation
4. b) Euler number < 1 → Save each patch, if the patches overlap, merge them into a bigger patch. (we were running the window check with overlap 50%)
5. For each (merged) patch → Project the component onto a 2d grid by interpolating it on a given number of coords. Perform smoothing, remove „over“-interpolation by doing flood-remove from the edges, thicken it with binary_dilation to 3 voxel and binary_fill_holes.
6. Compare the interpolated new component with the old component → if dice and coverage (only TP dice) high enough replace the inner part of the patch of old component with the new one.
7. If it dice or coverage score is too low, repeat step 4+ by using the next segmentation with a higher threshold AND for each component (with enough voxels) in the new segmentation of that patch (this way we removed merges of sheets).
8. Repeat until score high enough or removed.

Finally remove_small_objects again and pad the volume back to its original size.

This is how the different segmentations looked, as you can see, they "unmerge" by increasing the threshold:

Base

Base segmentation

0.5

T_low=0.5

0.6

T_low=0.6

0.7

T_low=0.7

result

Result

**Postprocessing history:**

It is too much right now, to explain how we ended up at the final method. I will probably write a comment/edit to explain it further. But the final method was basically implemented 1 day before the competition ended. Before that we had a very similar global interpolation pipeline (without patches basically). This would have scored 0.631, but it performed worse on public LB and CV for one reason: The label 2 issue. The global interpolation made the risk of a sheet dipping into label 2 territory way higher. I think the local method is superior anyway, but we didnt have any time to tune the hyperparameters and thresholds. And in the end I decided to use two local interpolation methods with different patch sizes.

Scores

But I think this is still an awesome outcome especially since I heavily realied on @pgeiger visualization tool throughout the whole competition. Without this contribution I would have probably given up. It helped me so often to understand what is going wrong/on. So thank you alot for this and a well deserved victory for your team!

Another special thank you to my team mate @nguyncdngs ! It was the first time I teamed up with someone and I didnt know what to expect. It was incredibly helpful and a joy to work with you. Thank you so much!

Feel free to ask any questions, I'll happily answere them in the comments!

Edit:

Here you can find the submission code for the global interpolation postprocessing, which scored the highest for us on private LB:

0.631 global interpolation solution notebook

And here our final submission for the private LB:

0.622 local interpolation solution notebook

It was heavily optimized by Claude (from ~3 minutes postprocessing per volume down to ~1 minute). The main difference to the described method above:

1. It checks the euler number for the whole component at once and completely replaces it if it detects a hole/tunnel.
2. After going completely through it once, it does another full postprocessing loop. This can be set higher than once, but doesnt improve it anymore, since it tries the same stuff again and again

The rest should be the same, if I dont forget anything.

This is the score of the ensembled models without the interpolation stuff:

Ensembled models score

Since other teams scored higher with just their base models, I'd be interested if this postprocessing would improve their score even further!

## How did the postprocessing idea come to life?

I came up with the idea to use interpolation to replace errorprone sheet segmentation pretty early on. I thought it is an easy way to score high, given the topo metric. Back then I didn't really understand how the topo metric works. I thought it would be enough to have 0 holes/tunnel (which is true) and the right number of components (wrong, since it is calculated with persistence homology, which means that the location of the components is also important).

So for each component: I extracted the voxel coords, used SVD to get the PCA, project it to (u,v) and height w, create a meshgrid which covers the u,v coords and interpolate over it with scipy.interpolate. The amount of coords used and the grid resolution have to be chosen carefully to limit processing time. In code:

```
coords = np.column_stack(np.nonzero(component))
coords_mean = coords.mean(axis=0)
U, S, Vt = np.linalg.svd(coords - coords_mean, full_matrices=False)
tangent1, tangent2 = Vt[0], Vt[1]
normal_guess = Vt[2]

uv_coords = (coords - coords_mean) @ np.column_stack([tangent1, tangent2])
w_coords = (coords - coords_mean) @ normal_guess

if len(coords) > 5000:
    indices = np.random.choice(len(coords), 5000, replace=False)
    uv_coords_sample = uv_coords[indices]
    w_coords_sample = w_coords[indices]
else:
    uv_coords_sample = uv_coords
    w_coords_sample = w_coords

u_min, u_max = uv_coords[:,0].min(), uv_coords[:,0].max()
v_min, v_max = uv_coords[:,1].min(), uv_coords[:,1].max()
u_padding = (u_max - u_min) * 0.05
v_padding = (v_max - v_min) * 0.05

grid_u, grid_v = np.meshgrid(
    np.linspace(u_min - u_padding, u_max + u_padding, num=grid_resolution),
    np.linspace(v_min - v_padding, v_max + v_padding, num=grid_resolution),
    indexing='ij'
)

try:
    w_grid = griddata(uv_coords_sample, w_coords_sample, (grid_u, grid_v), method='linear')
except:
    w_grid = griddata(uv_coords_sample, w_coords_sample, (grid_u, grid_v), method='nearest')

if np.any(np.isnan(w_grid)):
    mask = np.isnan(w_grid)
    w_grid_nearest = griddata(uv_coords_sample, w_coords_sample, (grid_u, grid_v), method='nearest')
    w_grid[mask] = w_grid_nearest[mask]
```

This approach has multiple problems:

- The interpolation ignores the boundaries of the original component. It creates a 2d sheet over the whole grid:

  interpolation

This could cause the interpolated component to reach into other components or create huge segmentations for small components. This was fixed by using flood-fill from edges (eventhough it is flood-removal in this case). It starts at each edge voxel in the 2d grid and checks if it is background or foreground in the original grid (left picture above). If is foreground -> stop for this voxel. If it is background, remove the voxel from the interpolated grid (right picture above) and check its neighbours with the same logic. In 2d space this removes interpolated background, which isn't inside of the sheet (a hole)!

flood-remove

- Sheet components which are close to each other might get merged by the interpolation since it doesnt perfectly replace the sheets. This is fixed by overthickening the components. Instead of 3 voxel thick components, it creates 5 voxel thick components. While merging the each components back into the same 3d volume, it checks for overlap and removes each overlapping voxel. This causes the components to only touch anymore. Now we erode each component back to 3 voxel thickness, which will unmerge them again!

erode

- The biggest problem. Components can't be interpolated with a single sheet component if they consist of multiple merged sheets. This was a huge problem and I was stuck for a long time. I tried other postprocessing methods and always went in circles. High logits threshold would cause the sheets to separate, but also worsen the overall segmentation heavily. Low logits threshold would merge the sheets again. Now the solution seems easy, we try to take the best from each world (segmentation). We detect merged sheets, by calculating a dice and coverage score for the interpolated component with the original component. If the score is high enough, the component is most likely a single sheet. So we keep this interpolation. If the score is low, it must be multiple merged sheets. So we throw away the interpolation and repeat the interpolation process on the same area with a higher threshold segmentation for each component in that area. This works, since the higher threshold segmentation will always be a part of the lower threshold segmentation. Here an example of the workflow with 2 alternative thresholds (0.5 and 0.7):

  workflow

In this example component 3 is multiple sheets merged into one. So another interpolation is started with threshold 0.5, which still detects one component and still scores low. So the process is repeated again. With threshold 0.7 it detects 3 components and can successfully interpolate each of them.

This approach can cause oversegmentation (multiple components for a single sheet), but it is better than keeping merged components with holes for the given metric.

# Supplemental Q&A

> **pp1985 · 2026-03-01T08:19:35.150000**

> I didn't really understand a lot of post processing steps which you did, how did you find out about these ideas ? did visualization help a lot in doing it ?

> **Marius Heuser · 2026-03-01T09:02:37.607000**
>
> I also noticed, that the postprocessing steps aren't as clear as I thought, while writing them. I will update the explanation today. It is way easier to understand the steps and problems if you see them visualized. And yes, visualization helped alot, but also just time. I was stuck in this post processing 3-4 times, but after trying different stuff and coming back to it, I finally found a way to solve each problem.

> **Marius Heuser · 2026-03-01T12:00:26.183000**
>
> I updated the writeup at the end with some more details, if you have any questions. Feel free to ask, I'll happily answere them :)

> **Arpit1Bansal · 2026-03-04T10:05:51.873000**
>
> Hey, can you share your train/val curves while training? What things let you decide, that this needs to train for 1000 epochs. which is a large number. And yes congrats to the whole team.

> **Duong Nguyen · 2026-03-04T11:38:35.250000**
>
> Hi @arpit1bansal this is the train/val curves
>
> `What things let you decide, that this needs to train for 1000 epochs. which is a large number.`
>
> 1000 epochs is the default setting of nnU-Net. At first, I used the default setting as a baseline, but the training took too long, so I decided not to increase the number of epochs